MANCHESTER 1824
The University of Manchester

MATH49111/69111: Scientific Computing

Expressions

Matthias Heil

M.Heil@maths.manchester.ac.uk

Expressions

```
#include <iostream>

int main()
{
    double sum = 0.0;  // Set up variables
    int n_terms = 100, i=1;

    while (i<=n_terms) /* Loop and sum terms */
    {
        sum += 1.0/(i*i);
        i++;
    }

    std::cout << "Sum of sequence after " << n_terms
              << " terms = " << sum << std::endl; // output result

    return 0;
}
```

Expressions

```
#include <iostream>

int main()
{
    double sum = 0.0;  // Set up variables
    int n_terms = 100, i=1;

    while (i<=n_terms) /* Loop and sum terms */
    {
        sum += 1.0/(i*i);
        i++;
    }

    std::cout << "Sum of sequence after " << n_terms
              << " terms = " << sum << std::endl; // output result

    return 0;
}
```

Q: Has anybody paid any attention to my style edicts and therefore spotted a crime in the above code?

Expressions

```
sum += 1.0/(i*i);
```

- *Expressions* are statements that typically perform calculations
- They are expressed in a mathematical syntax with:

  Operators, such as `+`, `-`, `*`, `=`, `==`, `+=`, `<<`, `++` (∼50 in total)

  Operands, which are variables (e.g. `i`, `sum`) and literals (e.g. `1.0`, `"Text"`)
- Operators have a *precedence* which determined the order they are evaluated (an extension of the familiar 'BODMAS' rule)
- Operators always **evaluate to** a value ($1 + 2$ evaluates to 3)
- Operators can also **change** the value of their operands:
  `a = b` changes the value of `a` to `b` (as well as evaluating to `b`)

Expressions

How do we evaluate the following expression?

```
sum += 1.0/(i*i);
```

- The operators (in order of precedence) are `()`, `/`, `*`, `+=`
- The highest-precedence operator `()` returns the value of the expression inside it, `i*i`, which we now evaluate:
  - The operator `*` returns the product of its two operands (here $\texttt{i}^2$)
- The next highest-precedence operator is `/`, which returns its left operand (`1.0`) divided by its right `(i*i)`
- The operator `+=` increments its left operand (`sum`) by its right,

$$\texttt{sum} \leftarrow \texttt{sum} + \frac{\texttt{1.0}}{\texttt{i}^2}$$

Expressions – another style edict!

If in doubt use brackets to make it absolutely clear what's intended!

- This statement doesn't need brackets at all:

```
int result = 10 + 5 * 2 - 8 / 4 + 3 * 3 - 6;
```

  Specifically, using operator precedence:

  - $\times$ and / before $+$ and - ("Punkt vor Strich" for the Germans amongst you...)
  - Left-to-right associativity for operators of the same precedence

  the compiler evaluates this as

$$\begin{aligned} \texttt{result} &= 10 + 5 \times 2 - 8/4 + 3 \times 3 - 6 \\ &= 10 + (5 \times 2) - (8/4) + (3 \times 3) - 6 \\ &= 10 + 10 - 2 + 9 - 6 \\ &= 21 \end{aligned}$$

- ...but any human reading this will much prefer the following version:

```
int result = ((10 + (5 * 2)) - (8 / 4)) + ((3 * 3) - 6);
```

Expressions – More operators

Some common operators, in order of decreasing precedence:

(See http://tinyurl.com/operator-precedence for a complete list)

Operator	*Evaluates to:*	*Name*
`a++`, `a--`	`a`, **and sets** `a` **to** `a+1`, `a-1`	postfix increment/decrement
`++a`, `--a`	`a+1`, `a-1`, **and sets** `a` **to** `a+1`, `a-1`	prefix increment/decrement
`!a`	true iff `a` is false	logical NOT
`*a`, `&a`	value-at address, address-of variable	dereference, address-of
`*`, `/`, `%`	$a \times b$, a/b, $a \bmod b$	multiply, divide, remainder
`a+b`, `a-b`	$a - b$, $a + b$	addition, subtraction
`<`, `>`, `<=`, `>=`	true iff $a < b$, $a > b$, $a \leq b$, $a \geq b$	$<$, $>$, $\leq$, $\geq$
`a == b`	true iff `a` equal to `b`	equality
`a != b`	true iff $a \neq b$	inequality
`a && b`	true iff `a` and `b` true	logical AND
`a || b`	true iff `a` or `b` true	logical OR
`a = b`	`b`, **and sets** `a` **to** `b`	assignment
`a += b`	`a+b`, **and sets** `a` **to** `a+b`	assignment by sum
		(`-=`, `*=`, `/=`, `%=`, similarly)

Expressions – Equality

Be careful of the two 'equals' operators:

Operator	*Evaluates to:*	*Name*
`a == b`	true iff `a` equal to `b`	equality
`a = b`	`b`, **and sets** `a` **to** `b`	assignment

- Is `a = b` equivalent to `b = a`?
- Is `a == b` equivalent to `b == a`?

A common bug:

Q: What will this code do? [Haven't done if statements yet, but you know what I mean...]

```
int a=0, b=0;
if (a=b)
{
 std::cout << "a and b are the same" << std::endl;
}
else
{
 std::cout << "a and b are different"<< std::endl;
}
```

A common bug:

Q: What will this code do? [Haven't done if statements yet, but you know what I mean...]

```
int a=0, b=0;
if (a=b)
{
 std::cout << "a and b are the same" << std::endl;
}
else
{
 std::cout << "a and b are different"<< std::endl;
}
```

Q: ...and what happens if you initialise `a` and `b` to 1?

Non-mathematical expressions

```
std::cout << "Sum of sequence after " << n_terms
          << " terms = " << sum << std::endl;
```

- Expressions are used for non-mathematical purposes too
- To display text to the screen, we write an expression with the `std::cout` object and insertion operator `<<`.
- We can display text strings in quotes (`"text"`) and variables of many types including `int` (`n_terms`) and `double` (`sum`)
- To move onto the next line of text, we sent another built-in object, `std::endl`
- The code above displays the text:

```
Sum of sequence after 100 terms = 1.63498
```

Libraries

```
#include <iostream>
```

- `std::cout` and `std::endl`, along with many other objects for common tasks, are part of the C++ *standard library*
- We include a part of this library with the code `#include <header_name>` (typically at the top of our code)
- Standard library *headers* include
  - `<iostream>` (Displaying text on the screen, keyboard input)
  - `<fstream>` (Loading and saving files to disk)
  - `<vector>` (Store list of data), see also `<list>`, `<map>` etc. for other data structures
  - `<cmath>` (Common mathematical functions, e.g. $\sqrt{x}$)
- There are **many** third-party C++ libraries that implement more specialised algorithms – or you can write your own.